



Современные архитектуры ИС



Лекция 3
Обучение на GPU



План лекции

1. **Анатомия GPU**
2. Иерархия памяти
3. Метрики производительности GPU
4. Типовые операции нейронных сетей на GPU
5. Программная модель CUDA

Почему мы используем GPU?

Один линейный слой записывается следующим образом:

$$y = xW + b$$

При перемножении или сложении матриц и векторов каждый элемент результата вычисляется независимо от других.

При вычислении на CPU, эта информация нам почти ничего не дает

Однако, на GPU мы можем проводить вычисления параллельно.

Почему мы используем GPU?

CPU позволяет проводить вычисления с минимальной **задержкой**, то есть минимизирует время выполнения последовательности операций

GPU максимизирует **пропускную способность** - количество операций в единицу времени.

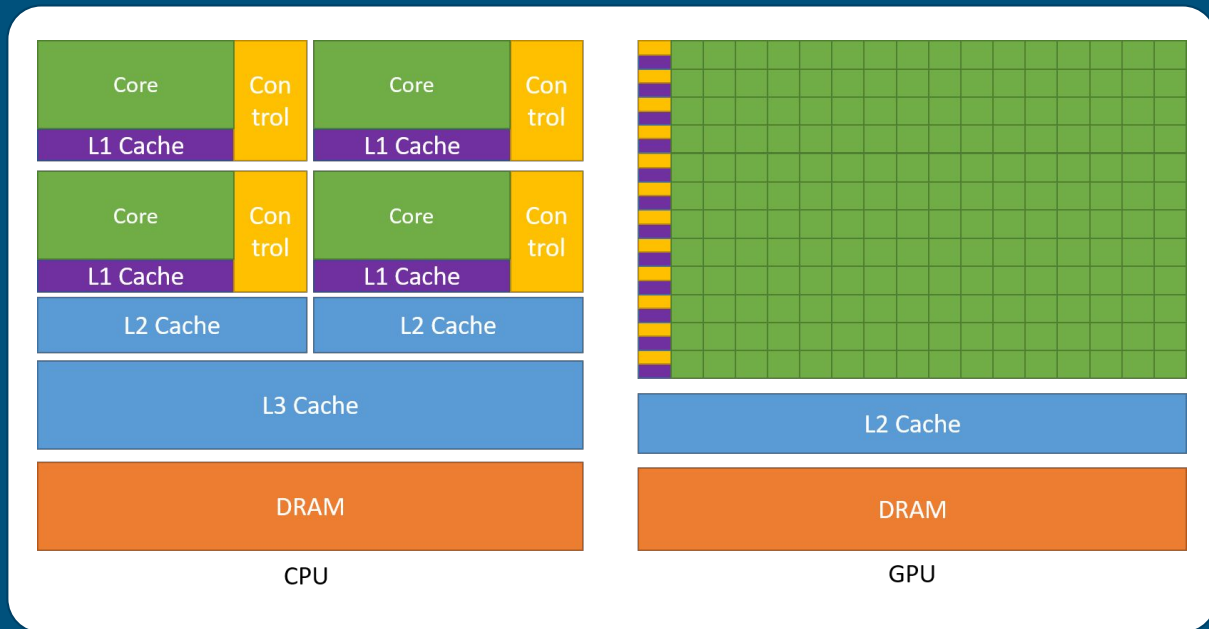
Почему мы используем GPU?

CPU позволяет проводить вычисления с минимальной **задержкой**, то есть минимизирует время выполнения последовательности операций

GPU максимизирует **пропускную способность** - количество операций в единицу времени.

Но не все модули нейронной сети так легко распараллелить.

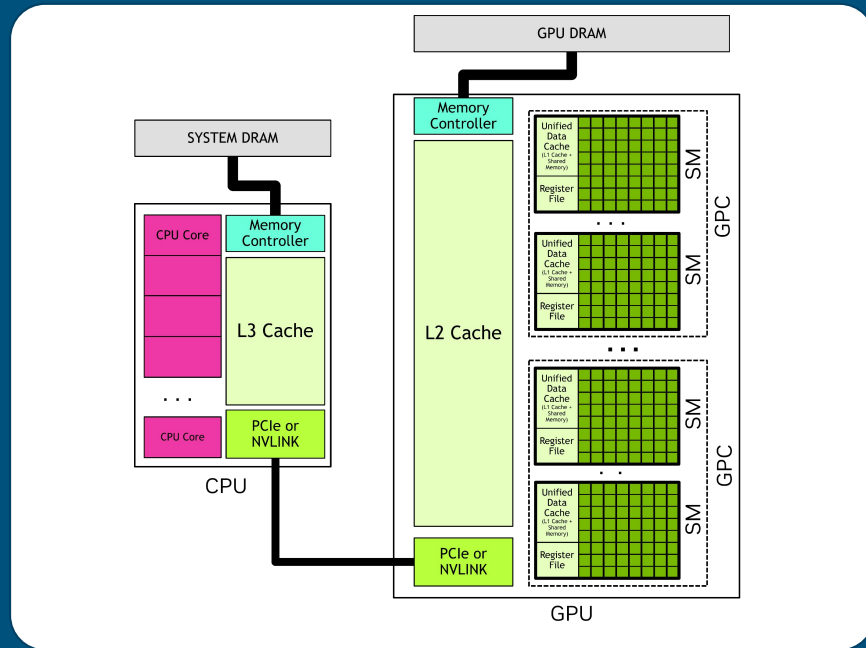
Почему мы используем GPU?



Streaming Multiprocessors

GPU состоит из большого числа вычислительных блоков, которые называются Streaming Multiprocessors (SM).

В некотором смысле, SM — это как ядро в CPU.



Потоки и варпы

Когда мы запускаем вычисления на GPU, задача разбивается на большое количество потоков. Например, при сложении двух векторов, один поток может отвечать за вычисление одного элемента:

$$c_i = a_i + b_i$$

Однако, такие операции не вычисляются полностью независимо. Они группируются в варпы.

Потоки и варпы — программные единицы, они не имеют “физического” представления на кристалле GPU.

SIMT

Варп — это группа из 32 потоков, которые исполняются вместе.

Варп является базовой единицей исполнения инструкций на GPU.

Одна инструкция применяется сразу ко всем потокам варпа.

SIMT

Варп — это группа из 32 потоков, которые исполняются вместе.

Варп является базовой единицей исполнения инструкций на GPU.

Одна инструкция применяется сразу ко всем потокам варпа.

Все потоки внутри варпа в каждый момент времени или исполняют одну и ту же инструкцию или не работают.

Такой принцип называется **SIMT** — Single Instruction, Multiple Threads.

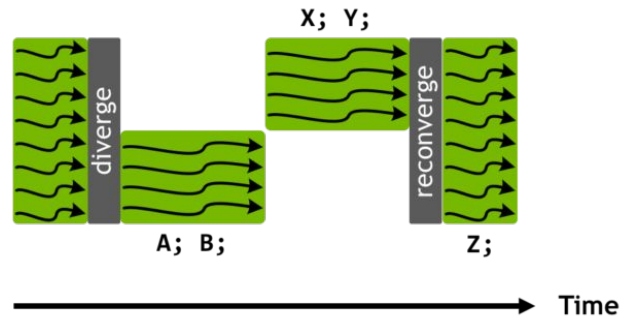
Расхождение варпа (warp divergence)

Если в программе есть логическое ветвление, то может появиться несколько версий инструкций.

Все эти версии будут выполняться по очереди.

Это называется расхождением варпа

```
if (groupThreadID.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```



CUDA-ядра

CUDA-ядра — это ALU внутри SM.

Существуют версии под разные типы данных.

Для нас интересны типы с плавающей точкой: fp32, fp16, fp8, fp4

А также версии под целочисленные значения



Тензорные ядра

Тензорные ядра —
специализированные ядра для
матричных умножений.

Обычно, на GPU их значительно
меньше, чем CUDA-ядер, обычно от 1
до 4 на SM



Варп-планировщики

Внутри SM есть аппаратные блоки, которые выбирают, какой варп будет запущен — варп-планировщики.

Варпы не запускаются одновременно, поскольку часть может быть готова раньше, а другая часть — позже.



Варп-планировщики

Варпы делятся на активные и неактивные.

Активные варпы — это те, что заняли ресурсы.

Максимально на SM может быть запущено одновременно ограниченное число варпов (это число зависит от архитектуры, например 32, 48 или 64).

Варп-планировщики

Варпы запускаются по очереди, поскольку одни, например, получили необходимые данные для работы, а другие — нет.

CPU в подобной ситуации минимизирует задержку (latency) за счёт сложных кэшей, предсказаний ветвлений и других инженерных хитростей.

В GPU же задача варп-планировщика — запускать варпы таким образом, чтобы уменьшить задержку выполнения всей программы посредством перекрытия работы одного варпа — другим варпом.

План лекции

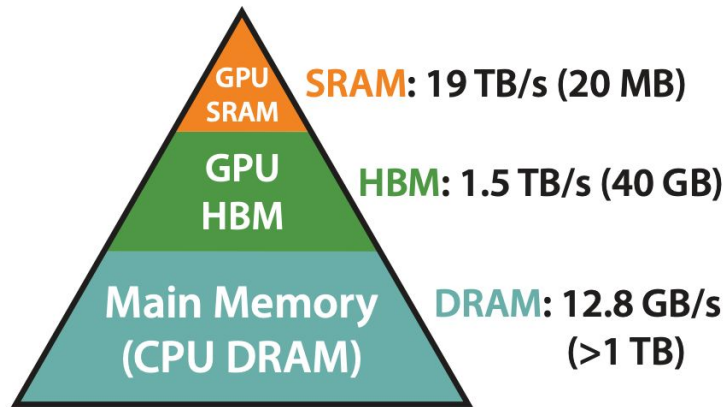
1. Анатомия GPU
2. **Иерархия памяти**
3. Метрики производительности GPU
4. Типовые операции нейронных сетей на GPU
5. Программная модель CUDA

Иерархия памяти

Данные надо где то хранить.

Обращение к памяти — не мгновенно. Невозможно сделать память одновременно и самой быстрой и максимально объемной.

Появляется иерархия: чем память ближе к вычислениям, тем она быстрее, но меньше.

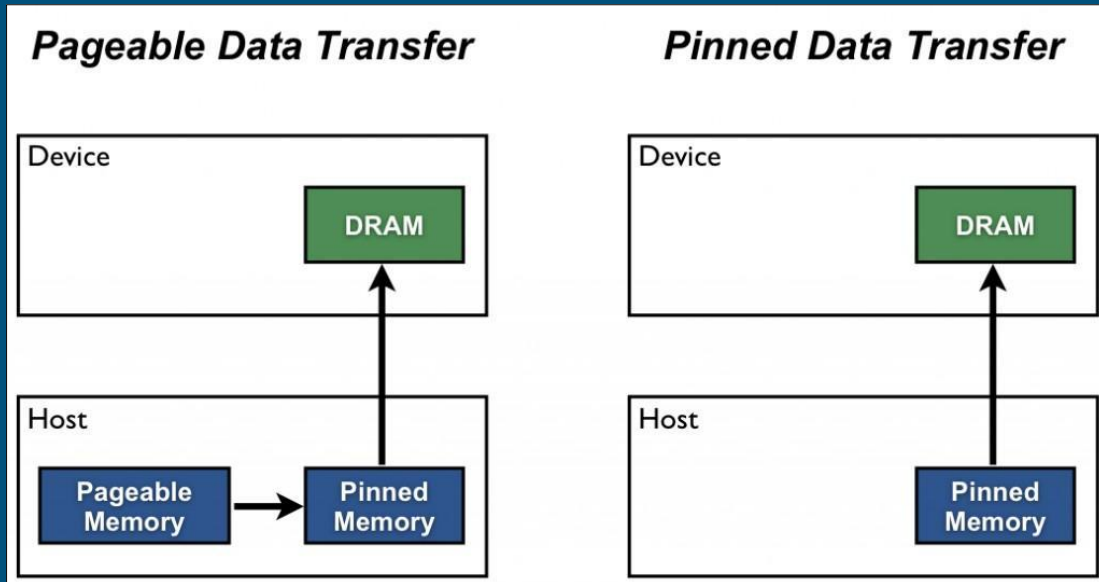


**Memory Hierarchy with
Bandwidth & Memory Size**

H2D обмен данными

Наиболее объемная и самая медленная память — память диска.

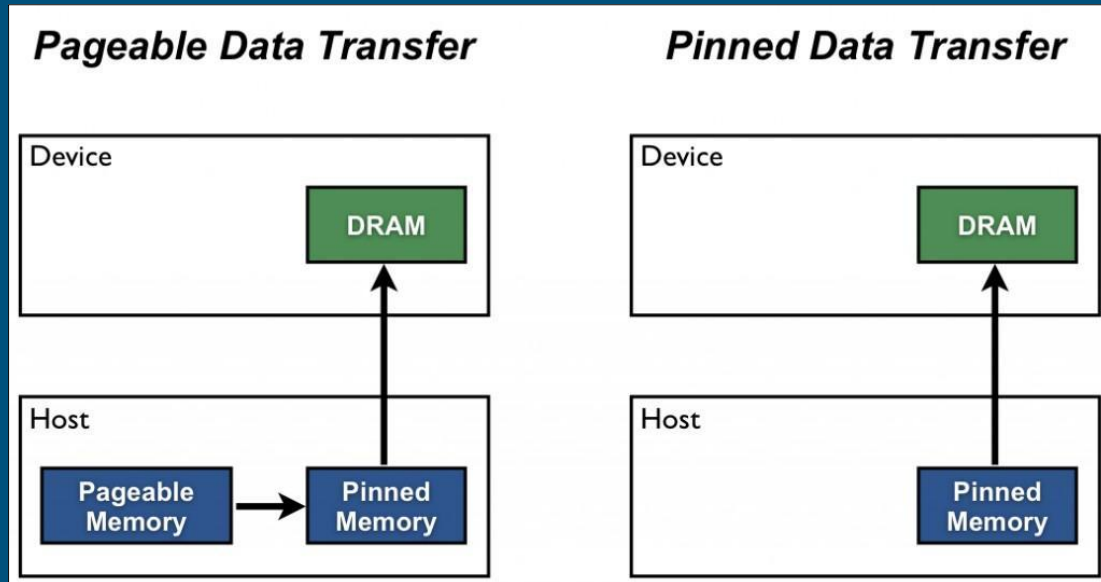
После диска, данные попадают в оперативную память хоста, где делятся на страницы



H2D обмен данными

Копирование данных с хоста на девайс не бесплатное.

Память хоста управляется с CPU. Для CPU удобно работать со страницам, а для GPU — с закрепленной памятью.



Глобальная память (GMEM)

Глобальная память — основная, самая объемная и самая медленная память GPU (хотя обращение к ней значительно дороже обращения к памяти хоста).

Глобальная память (GMEM)

Глобальная память — основная, самая объемная и самая медленная память GPU (хотя обращение к ней значительно дороже обращения к памяти хоста).

Часто задача оптимизации программы на GPU будет заключаться в оптимизации доступа к глобальной памяти.

Глобальная память (GMEM)

Глобальная память — основная, самая объемная и самая медленная память GPU (хотя обращение к ней значительно дороже обращения к памяти хоста).

Часто задача оптимизации программы на GPU будет заключаться в оптимизации доступа к глобальной памяти.

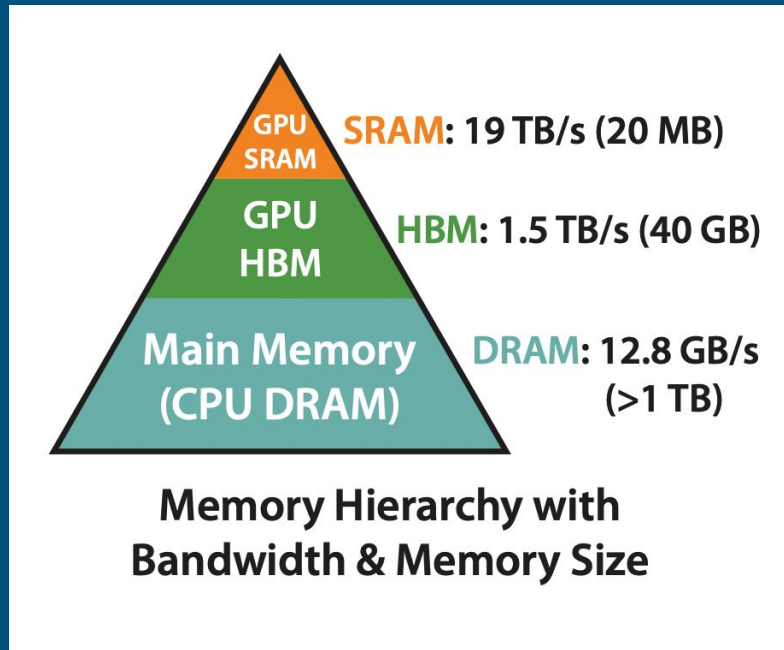
Через GMEM также происходит общение между разными программами на одном девайсе и между несколькими девайсами.

Разделяемая память (SMEM)

SMEM — следующий уровень иерархии.

Она намного меньше GMEM, но и доступ быстрее на несколько порядков.

SMEM позволяет обмениваться потокам одного варпа.

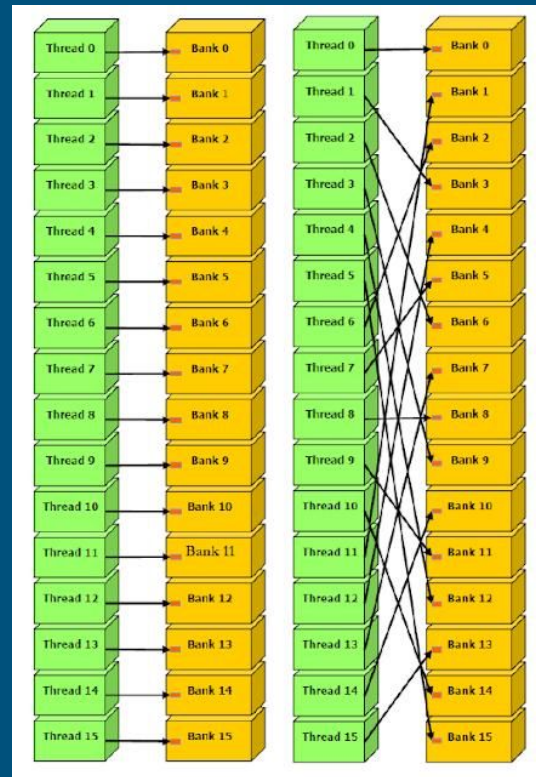


Конфликт банков

SMEM физически разбита на несколько банков.

Если несколько потоков одновременно обращаются к адресам одного банка, то доступ может выполняться медленнее.

Такая проблема называется **конфликт банков**.



Регистровая память (RMEM)

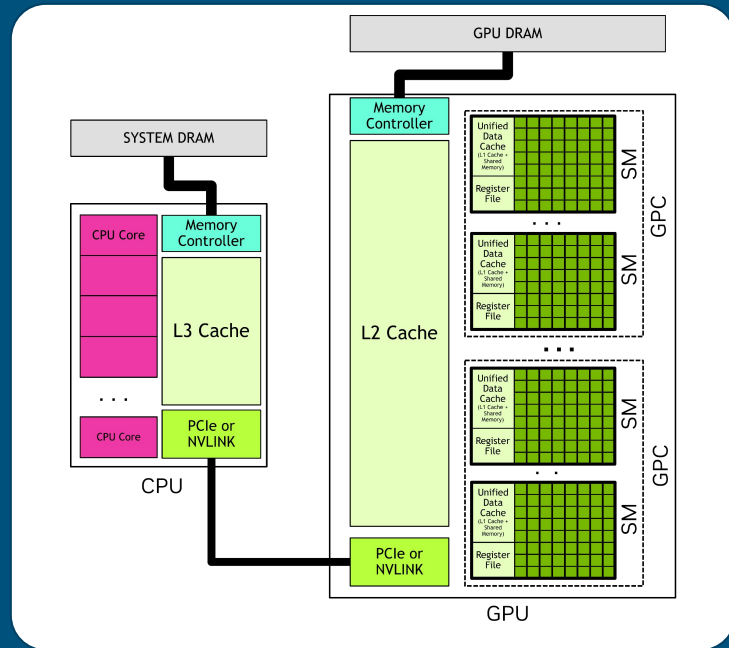
Самая быстрая память в иерархии — регистровая память. Доступ к ней выполняется практически мгновенно: за 1-2 такта.

Операции выполняются над данными, находящимися в регистрах. Данные в регистрах доступны только своему потоку.

Регистровая память (RMEM)

Регистровая память представлена в архитектуре как Register File. Register File разбит на определенное количество ячеек по 32 бита — регистровые ячейки.

Ресурсы регистров ограничены. Если регистров не хватает, компилятор будет использовать медленную local memory.



L1 и L2 кэш

L1 кэш находится внутри SM. Он кэширует часть обращений к глобальной памяти. На современных видеокартах NVIDIA L1 кэш и SMEM часто используют общий физический ресурс.

Соотношение размеров L1 кэша и SMEM не фиксировано. Чем больше используется SMEM, тем меньше остается места под L1.

L2 кэш – общий для GPU. Через него проходят все обращения разных SM к глобальной памяти.

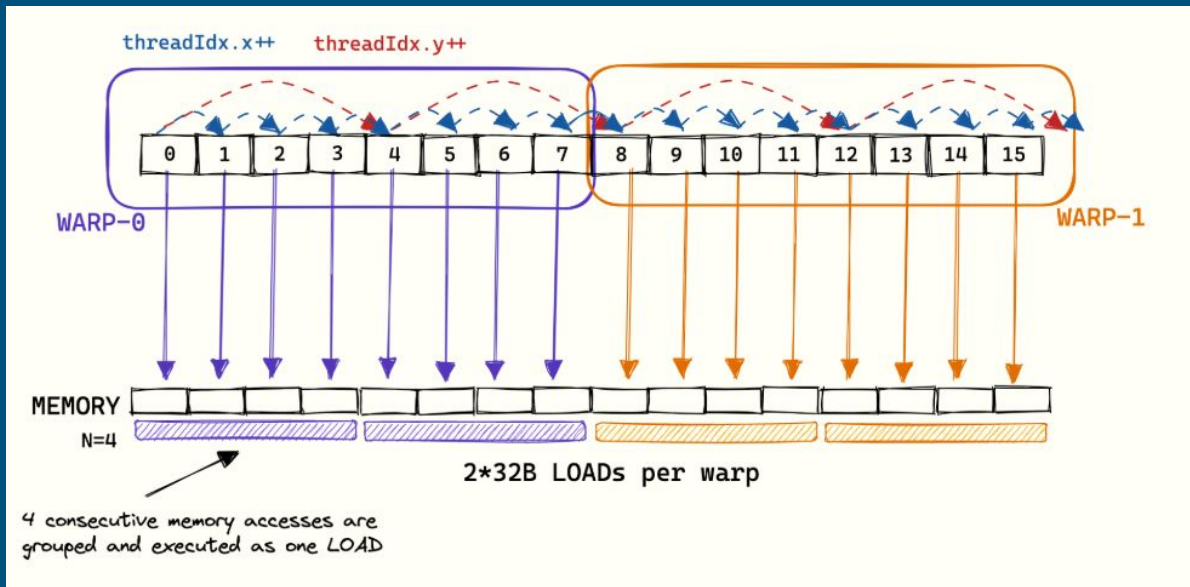
Согласованный доступ к памяти

Потоки исполняются варпами.

Если потоки одного варпа обращаются к соседним ячейкам памяти, GPU может объединить эти обращения в одну транзакцию, что кратно снизит стоимость обращения к памяти.

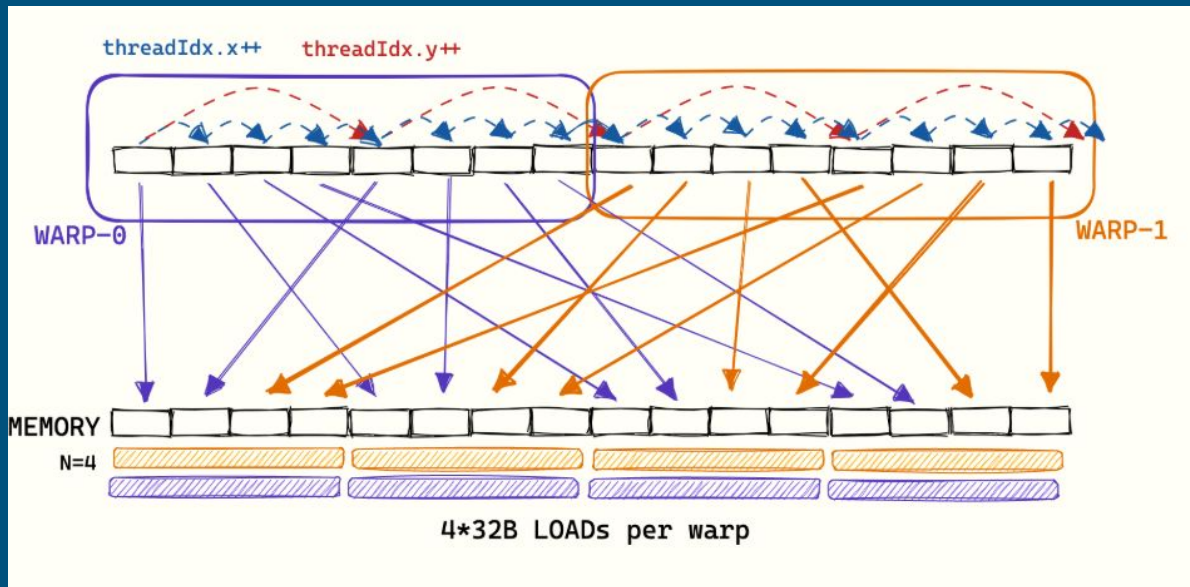
Согласованный доступ к памяти

Хорошо:

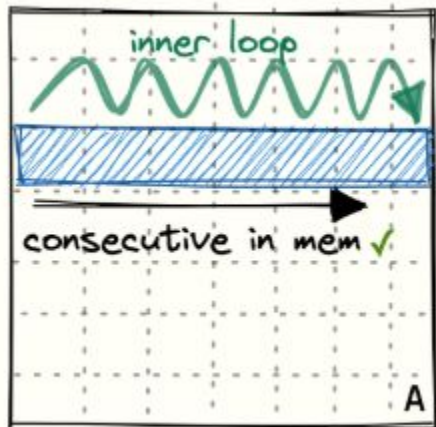


Согласованный доступ к памяти

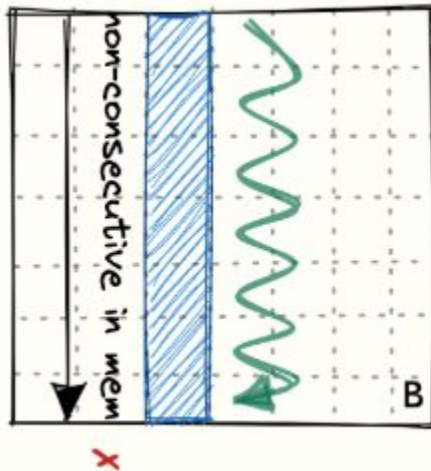
Плохо:



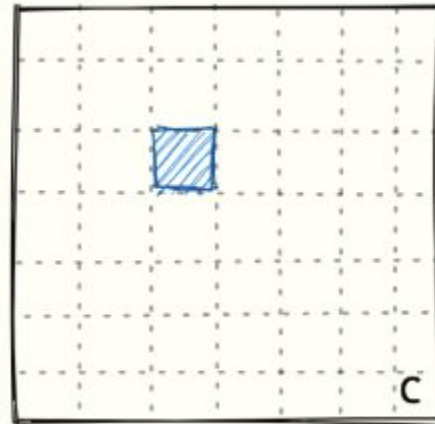
Согласованный доступ к памяти



⊗



=



План лекции

1. Анатомия GPU
2. Иерархия памяти
3. **Метрики производительности GPU**
4. Типовые операции нейронных сетей на GPU
5. Программная модель CUDA

Задержка и пропускная способность

Важно как то оценивать эффективность работы GPU и программы на GPU.

Первое что приходит в голову: давайте замерять end-to-end время работы. Эта метрика называется **задержкой** работы программы.

Задержка и пропускная способность

Важно как то оценивать эффективность работы GPU и программы на GPU.

Первое что приходит в голову: давайте замерять end-to-end время работы. Эта метрика называется **задержкой** работы программы.

Но GPU спроектированы для максимизации объема работы в единицу времени. Поэтому мы также должны оценивать **пропускную способность (throughput)**.

FLOP/s

Одна из основных характеристик вычислительной производительности — количество операций с плавающей точкой, которые GPU способен выполнять в единицу времени — FLOP/s.

Производитель указывает пиковое значение FLOP/s — теоретический предел. Важно отличать это значение от реально достижимого.

FLOP/s также сильно зависит от типа данных, в которых происходят вычисления.

Memory bandwidth

Для памяти же одна из основных метрик – пропускная способность памяти (memory bandwidth).

Если программа читает R байт и записывает W байт за время t , то пропускную способность можно оценить как $(R+W) / t$.

Арифметическая интенсивность

Какие то программы могут быть ограничены пропускной способностью в большей степени, а какие то – в меньшей. Для оценки введем метрику арифметической интенсивности:

$$\text{ArithmeticIntensity} = \frac{\text{FLOP}/s}{\text{MemoryBandwidth}}$$

Упростим:

$$\text{ArithmeticIntensity} = \frac{\frac{\text{FLOPs}}{t}}{\frac{W+R}{t}} = \frac{\text{FLOPs}}{W + R}$$

Roofline модель

Для грубой оценки максимальной производительности используется roofline-модель:

$$\text{Performance} \leq \text{ArithmeticIntensity} \cdot \text{MemoryBandwidth}$$

$$\text{Performance} \leq \text{PeakFLOP/s}$$

иначе говоря:

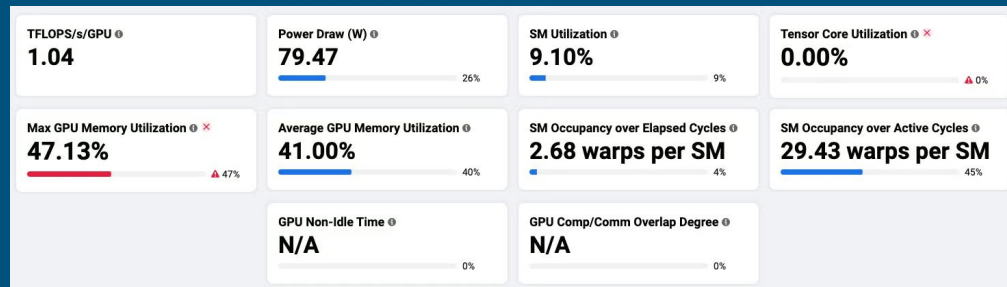
$$\text{Performance} \leq \min(\text{ArithmeticIntensity} \cdot \text{MemoryBandwidth}, \text{PeakFLOP/s})$$



Осцирансу

Осцирансу позволяет оценить,
насколько эффективно
нагружены SM

$$\text{Осцирансу} = \frac{W_{\text{active}}}{W_{\text{max}}}$$



Пример подробной оценки использования ресурсов
программой в MAIProf

План лекции

1. Анатомия GPU
2. Иерархия памяти
3. Метрики производительности GPU
4. **Типовые операции нейронных сетей на GPU**
5. Программная модель CUDA

Поэлементные операции

Самый простой тип – поэлементные операции. Например, сложения тензоров:

$$c_i = a_i + b_i$$

или применения поэлементных функций активации:

$$y_i = \max(0, x_i)$$

Поэлементные операции обладают очень низкой арифметической интенсивностью.

Редукции

В нейронных сетях часто встречаются редукции — операции, которые преобразуют массив элементов в скаляр.

В отличие от поэлементных операций, редукции параллелизуются сложнее и имеют низкую арифметическую интенсивность.

GEMV

GEMV-операция — это умножение вектора на матрицу. Например преобразование одного вектора с помощью линейного слоя:

$$y = xW$$

Подобная операция часто встречается при инференсе трансформера и использованием KV-кэша

GEMM

GEMM — это операция матричного умножения, самая важная операция для нейронных сетей, поскольку описывает линейный слой:

$$Y = XW$$

GEMM

GEMM — это операция матричного умножения, самая важная операция для нейронных сетей, поскольку описывает линейный слой:

$$Y = XW$$

Каждый элемент Y мы можем считать параллельно. В наивном случае, один поток вычисляет скалярное произведение строки на столбец:

$$Y_{ij} = \sum_{k=1}^D X_{ik} W_{kj}$$

GEMM

GEMM — это операция матричного умножения, самая важная операция для нейронных сетей, поскольку описывает линейный слой:

$$Y = XW$$

Каждый элемент Y мы можем считать параллельно. В наивном случае, один поток вычисляет скалярное произведение строки на столбец:

$$Y_{ij} = \sum_{k=1}^D X_{ik} W_{kj}$$

Такой подход имеет крайне низкую арифметическую интенсивность и требует оптимизации

Нормализация

Нормализации используются во всех современных архитектурах.
Рассмотрим на примере LayerNorm:

$$y_j = \gamma_j \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_j \quad \mu = \frac{1}{D} \sum_{j=1}^D x_j \quad \sigma^2 = \frac{1}{D} \sum_{j=1}^D (x_j - \mu)^2$$

LayerNorm нормализация выполняется по одному токену независимо. Для каждого токена необходимо выполнить редукции: посчитать среднее и сумму квадратов для дисперсии.

Нормализация — memory-bound операция с большим количеством синхронизаций

План лекции

1. Анатомия GPU
2. Иерархия памяти
3. Метрики производительности GPU
4. Типовые операции нейронных сетей на GPU
5. **Программная модель CUDA**

Технологический стек CUDA

На нижнем уровне находится драйвер. Он отвечает за взаимодействие с устройством.

Над драйвером находится CUDA Runtime — набор библиотек, инструментов и API. Предоставляет более удобный интерфейс для работы по сравнению с драйвером.

Над CUDA Runtime находится CUDA C/C++ — расширение языка C++, которое позволяет писать программы, работающие с CUDA Runtime. Такие программы компилируются с помощью NVCC.

CUDA-кernels

Функция, выполняемая на девайсе, называется CUDA кернел. Кернел описывает работу одного потока.

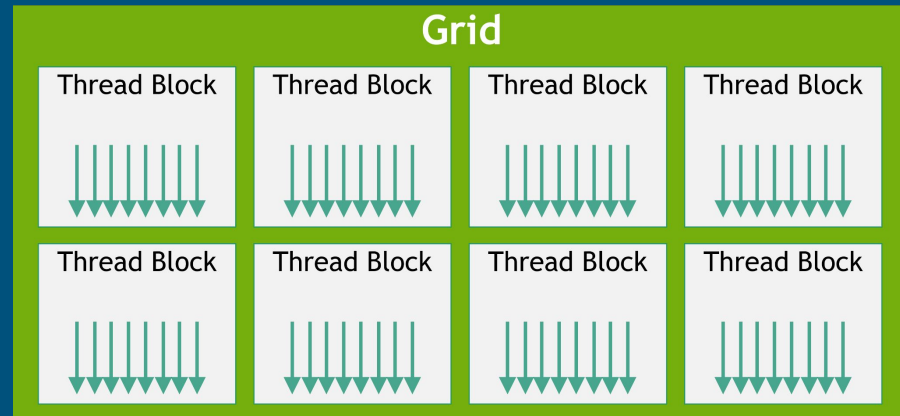
При запуске таких потоков много, каждый поток получает свой индекс и обрабатывает свой участок данных.

Запуск кернела — не бесплатный. Есть множество накладных расходов, называемых **kernel launch overhead**.

Threads, Blocks, Grid

Вычисления в CUDA организованы иерархически:

Grid $\rightarrow$ Blocks $\rightarrow$ Threads



Индексация

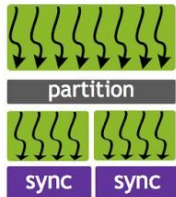
Каждый поток имеет свой индекс внутри блока, каждый блок имеет свой индекс внутри сетки.

Поскольку часто CUDA-кernels необходимо работать с многомерными массивами, для упрощения работы используется индексация потоков внутри блока и блоков внутри сетки по трём осям: x , y и z .

Синхронизации

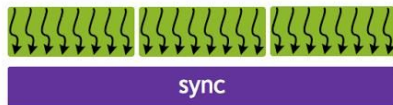
FLEXIBLE GROUPS

Define and
Synchronize Arbitrary
Groups of Threads

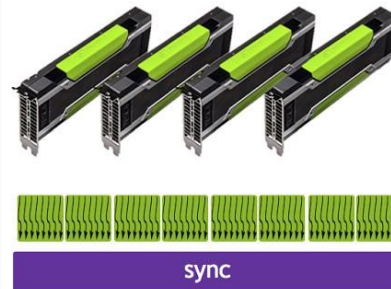


WHOLE-GRID SYNCHRONIZATION

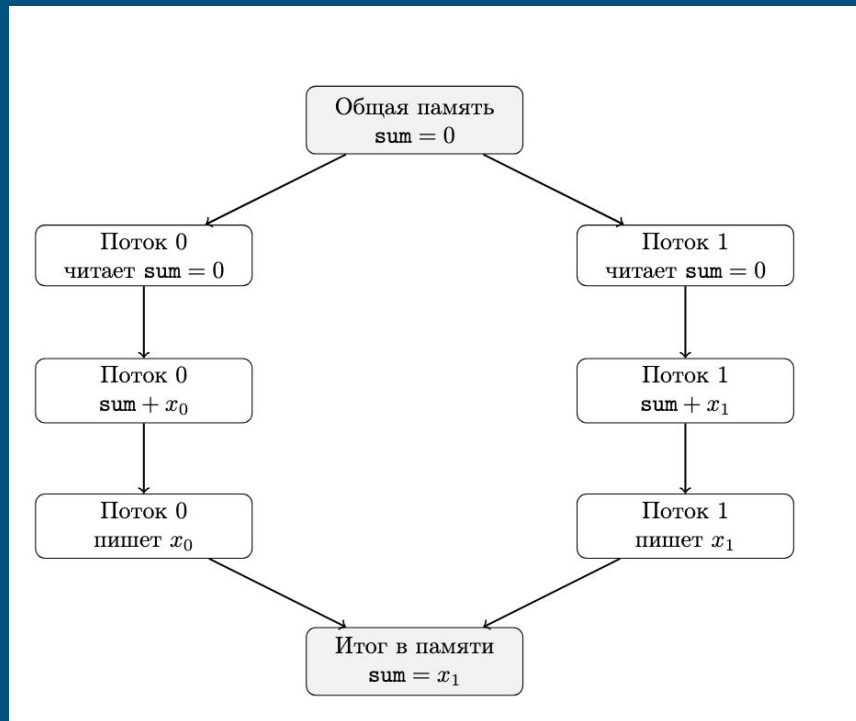
Synchronize Multiple
Thread Blocks



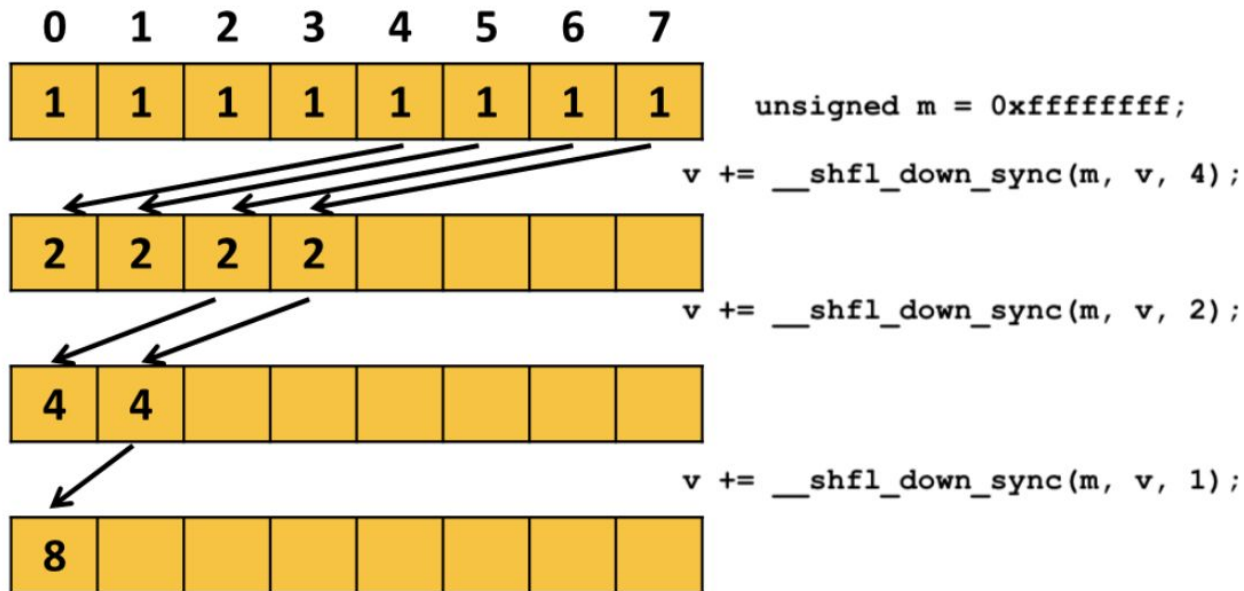
MULTI-GPU SYNCHRONIZATION



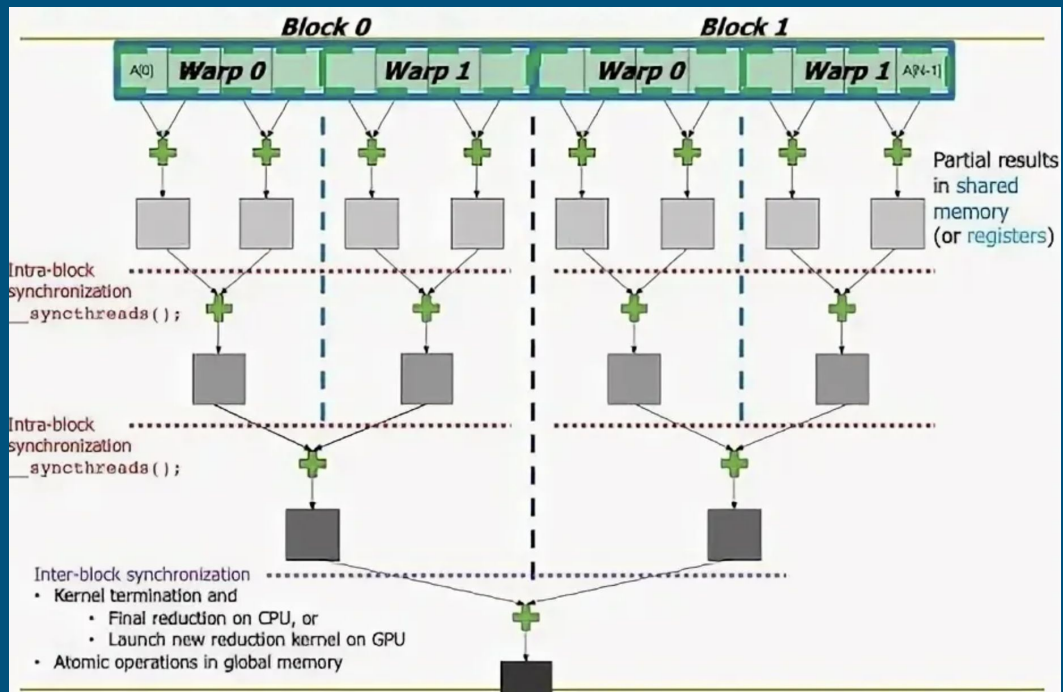
Race Conditions



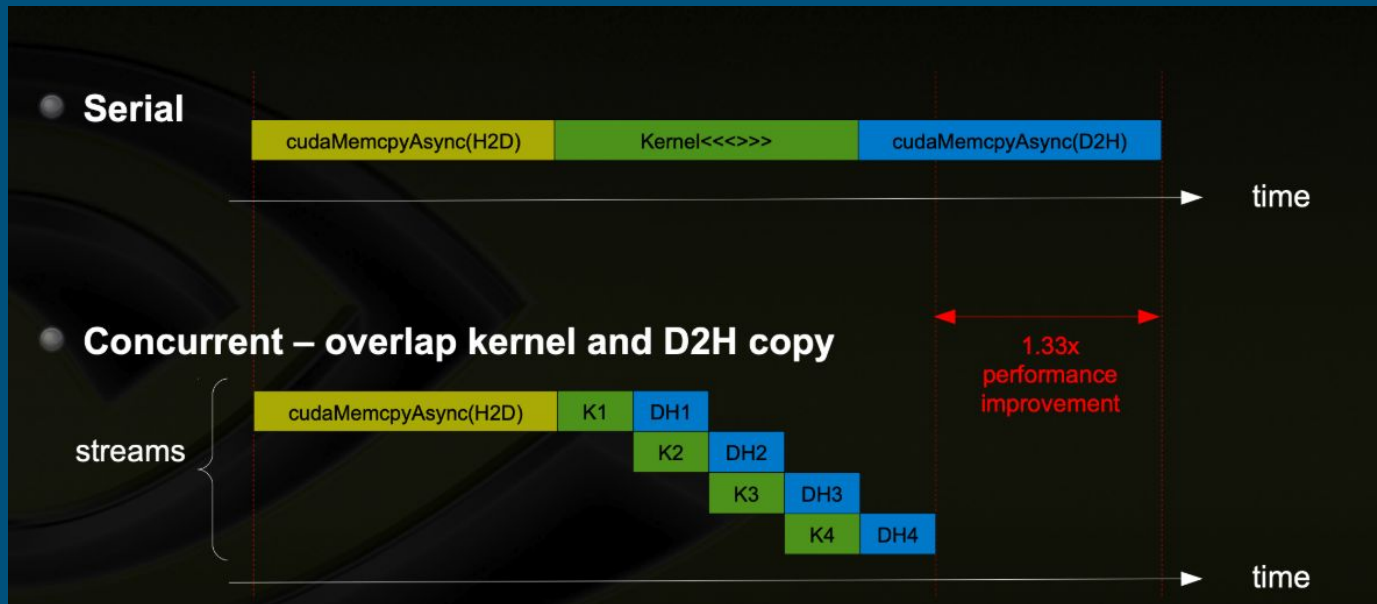
Варп-редукции



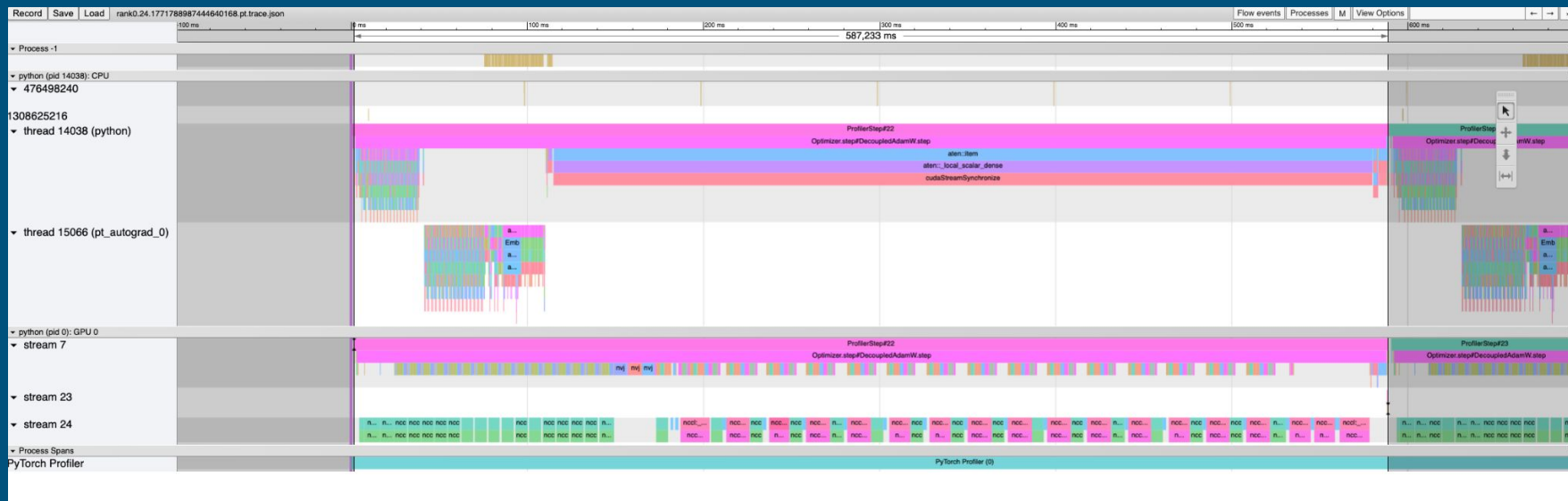
Варп-редукции



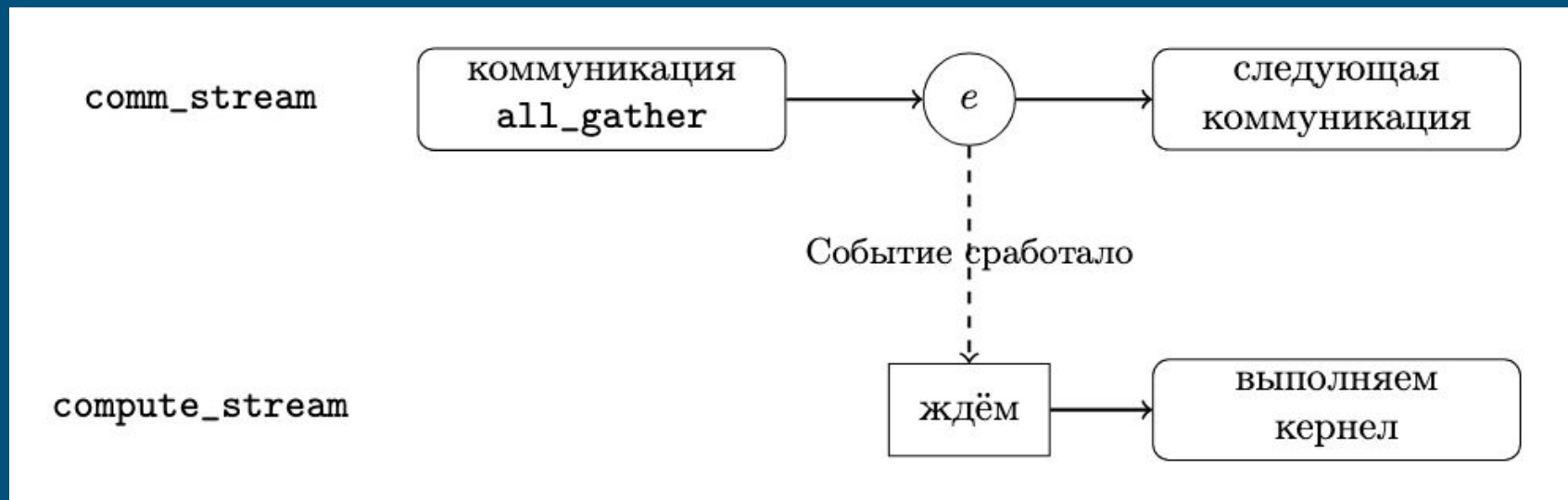
Стримы и асинхронность



Стримы и асинхронность



CUDA-события



Вопросы?
